Strategy

Strategy · Swap cart pricing (percent vs flat vs half) · Build with Waqar

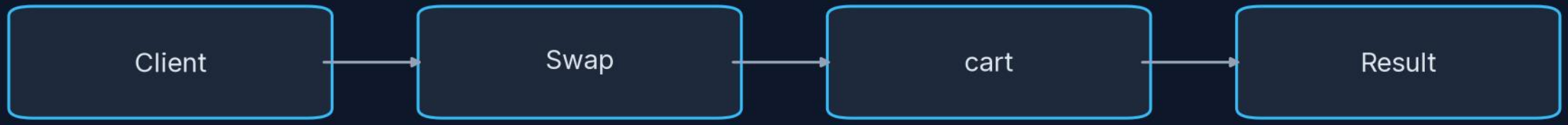
Today's Hook

- Problem: Swap cart pricing (percent vs flat vs half)
- Pattern: Strategy — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Swap cart pricing (percent vs flat vs half)
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Strategy: Swap cart pricing (percent vs flat vs half)

Real-World Scenario

- Running example: Swap cart pricing (percent vs flat vs half)
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.strategy;

import com.buildwithwaqar.patterns.gof.behavioral.strategy.*;
/** Problem: swap cart discount algorithms at checkout. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Strategy problem: cart pricing ===");
    System.out.println("10off=" + new Cart(PricingStrategy.percentageOff(10)).checkout(1000));
    System.out.println("flat=" + new Cart(PricingStrategy.flatOff(150)).checkout(1000));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.strategy;

import java.util.function.IntUnaryOperator;

interface PricingStrategy {
    int priceCents(int baseCents);

    static PricingStrategy percentageOff(int percent) {
        return base -> base - base * percent / 100;
    }

    static PricingStrategy flatOff(int cents) {
        return base -> Math.max(0, base - cents);
    }
}

/** Context that delegates to a pluggable pricing strategy. */
public final class Cart {
    private final PricingStrategy strategy;

    public Cart(PricingStrategy strategy) { this.strategy = strategy; }
```

Solution (2)

```
    /** Java 25 style: accept a function directly. */
    public Cart(IntUnaryOperator fn) { this.strategy = fn::applyAsInt; }

    public int checkout(int baseCents) { return strategy.priceCents(baseCents); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.strategy;

import com.buildwithwaqar.patterns.gof.behavioral.strategy.*;
/** Problem: swap cart discount algorithms at checkout. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Strategy problem: cart pricing ===");
    System.out.println("10off=" + new Cart(PricingStrategy.percentageOff(10)).checkout(1000));
    System.out.println("flat=" + new Cart(PricingStrategy.flatOff(150)).checkout(1000));
  }
}
```

Walkthrough Output

```
=== Strategy problem: cart pricing ===
10off=900
flat=850
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Strategy.

Related Patterns

- Contrast Strategy with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Strategy → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.